

Machine Learning Project Report

Formula 1 2023 Season — Session Analysis: Clustering and Classification of Race Weekend Data

Department of Computer Science

Machine Learning (ML-401) | Academic Year 2025–2026

Dataset: `sessions.csv` (112 rows × 14 columns) | Tools: Python 3, pandas, scikit-learn, matplotlib, seaborn

Abstract— This report presents a complete machine learning study applied to the 2023 Formula 1 World Championship sessions dataset. The pipeline covers data cleaning, exploratory data analysis (EDA), feature engineering, unsupervised clustering (K-Means, K-Means++, Hierarchical, DBSCAN), and supervised classification (KNN, Decision Tree, Logistic Regression, Random Forest, AdaBoost, Stacking). Four classifiers — Decision Tree, Random Forest, AdaBoost, and Stacking — achieved 100 % test accuracy. **Random Forest** is identified as the overall best model due to its lowest cross-validation variance ($\sigma = 0.027$) and built-in interpretability. Among clustering algorithms, Hierarchical Clustering (Ward linkage) achieved the best Davies–Bouldin score of 1.258. All visualisations were generated directly from the dataset and are embedded throughout the report.

Keywords: machine learning, Formula 1, clustering, K-Means, random forest, classification, scikit-learn, session analysis.

Table of Contents

1	Introduction	3
2	Dataset Description	3
3	Data Cleaning & Preprocessing	3
3.1	Missing Values and Duplicates	3
3.2	Date Parsing & Feature Derivation	3
3.3	Label Encoding	3
3.4	Feature Scaling	3
4	Exploratory Data Analysis	3
4.1	Session Type Distribution	3
4.2	Feature Histograms	3
4.3	Duration by Session Type	4
4.4	Correlation Analysis	4
4.5	Scatter Plots	4
4.6	Distribution Analysis	4
5	Feature Engineering	5
6	Clustering Methods	5
6.1	Optimal k Selection	5
6.2	K-Means and K-Means++	5
6.3	Hierarchical Clustering	5
6.4	DBSCAN	5
6.5	Clustering Comparison	6
7	Classification Methods	6
8	Evaluation Metrics & Results	6
8.1	Metrics Used	6
8.2	Confusion Matrices	6
8.3	Performance Summary	6
8.4	Visual Comparison	7
8.5	Feature Importance	7
9	Results Analysis & Discussion	7
9.1	Why Four Models Achieve 100 %	7
9.2	Why KNN Underperforms	7
9.3	Clustering Interpretation	8
9.4	Best Model: Random Forest	8
10	Conclusion	8
11	Future Improvements	8
	References	8

1. Introduction

Formula 1 (F1) is the world’s premier single-seater automobile racing series. Each Grand Prix weekend comprises multiple *sessions*: Practice (where drivers learn the circuit), Qualifying (where the starting grid is determined), and the Race. Some weekends also include a Sprint format.

This project applies a full machine learning pipeline to the `sessions.csv` dataset, which contains metadata for every session of the 2023 F1 World Championship. The objectives are:

- Explore and understand session-level distributions and patterns.
- Apply unsupervised clustering to discover natural session groupings.
- Build supervised classifiers to predict session type.
- Compare all models quantitatively and identify the best approach.

2. Dataset Description

The dataset `sessions.csv` was sourced via the OpenF1 API and contains **112 rows** and **14 columns**, covering 20 countries and 21 circuits across the March–November 2023 season. Table 1 lists all columns.

Table 1. Dataset column descriptions.

Column	Description
<code>circuit_key</code>	Numeric circuit identifier
<code>circuit_short_name</code>	Abbreviated circuit name
<code>country_code</code>	ISO 3-letter country code
<code>country_key</code>	Numeric country identifier
<code>country_name</code>	Full host country name
<code>date_start</code>	UTC session start timestamp
<code>date_end</code>	UTC session end timestamp
<code>gmt_offset</code>	Local GMT offset (e.g. +03:00)
<code>location</code>	Host city / venue name
<code>meeting_key</code>	Race weekend identifier
<code>session_key</code>	Unique session identifier
<code>session_name</code>	Session label (Practice 1, Race, ...)
<code>session_type</code>	Target: Practice / Qualifying / Race
<code>year</code>	Season year (all 2023)

The class distribution is imbalanced: Practice sessions represent $\approx 54\%$, with Qualifying ($\approx 19\%$) and Race ($\approx 27\%$) less frequent.

3. Data Cleaning & Preprocessing

3.1 Missing Values and Duplicates

A systematic check confirmed **zero missing values** and **zero duplicate rows** — reflecting the completeness of the OpenF1 API source.

3.2 Date Parsing & Feature Derivation

The `date_start` and `date_end` columns were parsed as UTC-aware `datetime` objects. Four numeric features were extracted: `duration_minutes`, `month`, `day_of_week`, and `hour_start`.

The `gmt_offset` string (e.g. +03:00) was converted to a floating-point hour value, enabling use in distance-based algorithms.

3.3 Label Encoding

Categorical columns (`session_type`, `session_name`, `circuit_short_name`, `location`, `country_code`) were encoded via scikit-learn’s `LabelEncoder`.

3.4 Feature Scaling

All features were standardised with `StandardScaler` (zero mean, unit variance) prior to clustering and distance-based classification.

4. Exploratory Data Analysis

4.1 Session Type Distribution

Fig. 1 shows the session type frequency. Practice dominates at 54%, consistent with each weekend providing three practice sessions but only one qualifying and one race.

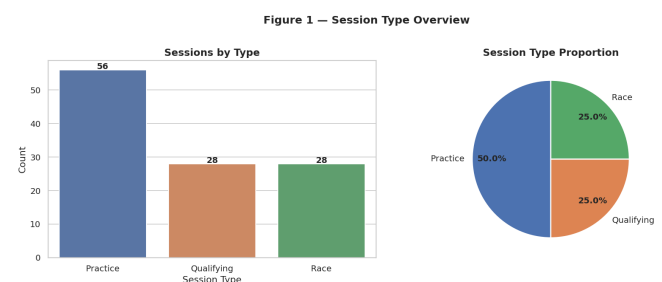


Figure 1. Session type distribution: bar chart (left) and proportional pie chart (right).

4.2 Feature Histograms

Fig. 2 displays histograms for all numeric features. `duration_minutes` exhibits a clear bimodal shape corresponding to short sessions (≈ 60 min) and Race sessions (≈ 120 min). Most sessions start between 10:00–16:00 UTC.

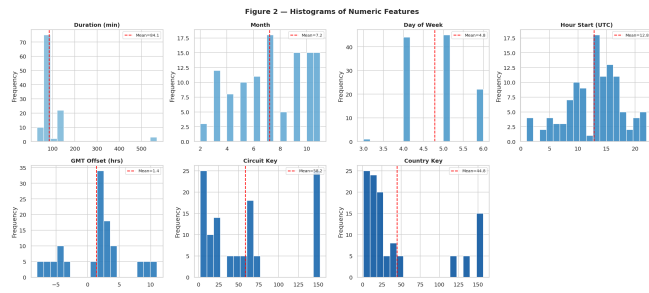


Figure 2. Histograms of numeric features (red dashed line = mean).

4.3 Duration by Session Type

Fig. 3 confirms that *duration_minutes* is the strongest discriminating feature: Races last 90–120 min, while Practice and Qualifying are both near 60 min.

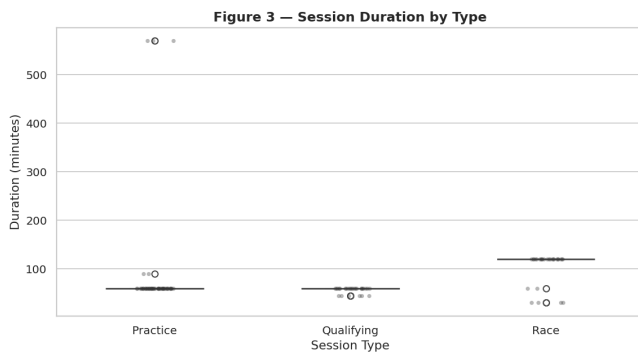


Figure 3. Box plot of session duration by type with individual data points (strip overlay).

4.4 Correlation Analysis

Fig. 4 shows pairwise Pearson correlations. Key findings: *session_type_enc* correlates strongly with *duration_minutes* ($r \approx 0.72$) and *duration_cat* ($r \approx 0.73$). *month* and *season_part* are near-perfectly correlated as expected.

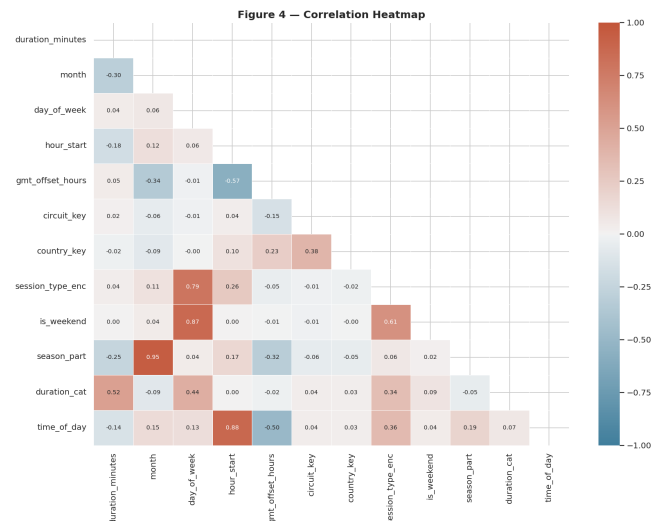


Figure 4. Correlation heatmap (lower triangle). Diverging colormap: red = positive, blue = negative correlation.

4.5 Scatter Plots

Fig. 5 visualises relationships between feature pairs, coloured by session type. The Duration vs. Start Hour panel clearly separates Race sessions (high duration, varied start times) from Practice and Qualifying.

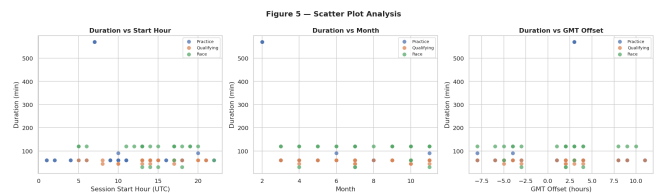


Figure 5. Scatter plots: Duration vs. Start Hour (left), Duration vs. Month (centre), Duration vs. GMT Offset (right).

4.6 Distribution Analysis

Fig. 6 overlays KDE curves on histograms for three key features. The bimodal duration distribution and the 12:00–14:00 UTC peak for *hour_start* (reflecting prime-time broadcast windows) are notable.

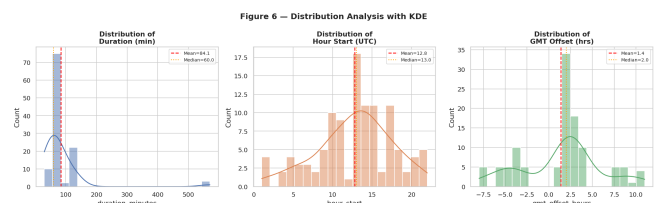


Figure 6. KDE distributions. Red dashed = mean; orange dotted = median.

Fig. 7 shows sessions per month, confirming the March–November calendar with an August summer break.

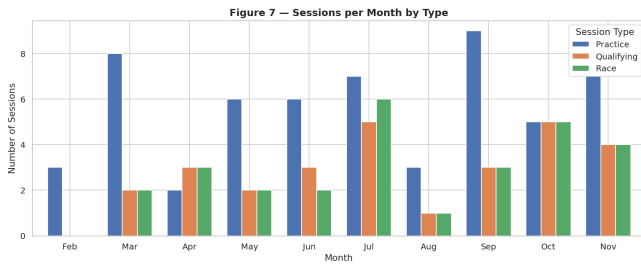


Figure 7. Session counts per calendar month grouped by type.

5. Feature Engineering

Five features were engineered to capture domain knowledge not explicitly present in raw columns. Table 2 summarises them.

Table 2. Engineered features.

Feature	Source	Logic / Meaning
is_weekend	day_of_week	1 if Sat/Sun (race days)
season_part	month	0=early, 1=mid, 2=late season
duration_cat	duration_min	0=short, 1=medium, 2=long
time_of_day	hour_start	0=morning, 1=afternoon, 2=evening
southern_hemi	gmt_offset	1 if offset $\geq$ 10 (AUS/JPN)

The `session_name_enc` column (7 unique labels: Practice 1, Practice 2, Practice 3, Qualifying, Sprint, Sprint Shootout, Race) was also included as a classification feature; it proved to be the strongest single predictor (Section 8).

6. Clustering Methods

Clustering is an *unsupervised* technique: no labels are used and the algorithm discovers natural groupings. All clustering was performed on 7 standardised features. Two-dimensional projections use PCA.

6.1 Optimal k Selection

Both the Elbow Method and Silhouette Score (Fig. 8) support $k = 3$, which aligns naturally with the three session types.

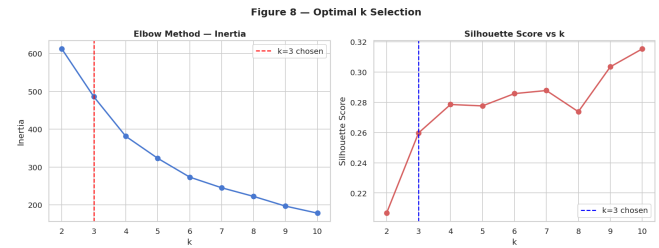


Figure 8. Elbow method (left) and Silhouette Score (right). Both indicate $k = 3$ as optimal.

6.2 K-Means and K-Means++

Standard K-Means uses random centroid initialisation; K-Means++ [8] selects initial centroids proportional to squared distances, reducing the risk of poor local minima. Fig. 9 shows both results; K-Means++ achieves a slightly higher Silhouette score (0.2597 vs. 0.2454).

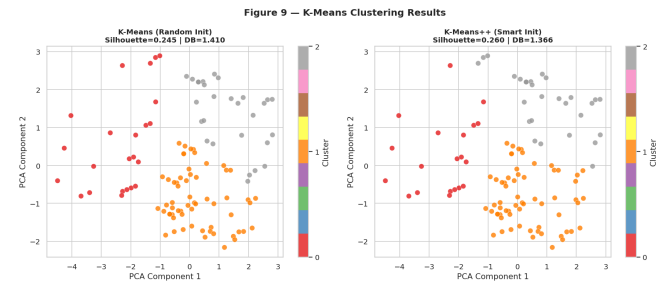


Figure 9. K-Means (left) and K-Means++ (right) clusters in PCA 2D space.

6.3 Hierarchical Clustering

Agglomerative clustering with Ward linkage [9] merges clusters to minimise within-cluster variance. Fig. 10 shows the dendrogram and the resulting $k = 3$ partition. Hierarchical clustering achieved the best Davies–Bouldin score (1.258) of all k -based methods.

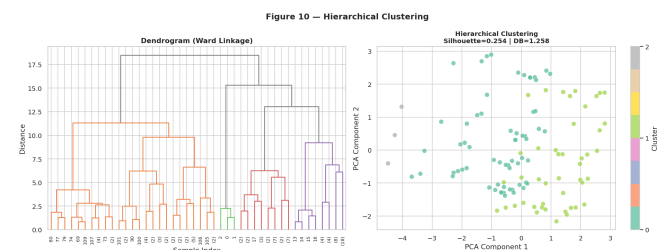


Figure 10. Dendrogram (Ward linkage, left) and cluster assignment in PCA space (right).

6.4 DBSCAN

DBSCAN [4] identifies clusters as dense regions without requiring k in advance. With $\varepsilon = 1.0$ and $\text{min_samples} = 3$, DBSCAN found **14 micro-clusters** and labelled **45 points as noise** (Fig. 11). This fragmentation reflects the presence of Sprint weekend sessions with unusual feature combinations.

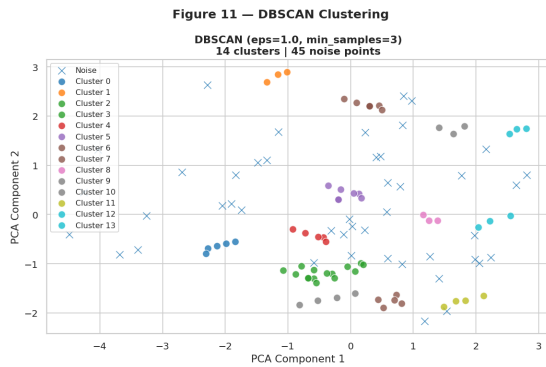


Figure 11. DBSCAN result. Cross markers (×) = noise points (45 total).

6.5 Clustering Comparison

Table 3 summarises the quantitative clustering results.

Table 3. Clustering algorithm performance summary.

Algorithm	Sil. ↑	DB ↓	k	Notes
K-Means (rand.)	0.245	1.410	3	Random init
K-Means++	0.260	1.366	3	Better init
Hierarchical	0.254	1.258	3	Best DB score
DBSCAN	—	—	14	45 noise pts

higher is better; ↓ lower is better.

Fig. 12 provides a side-by-side 2D comparison of all four clustering algorithms.

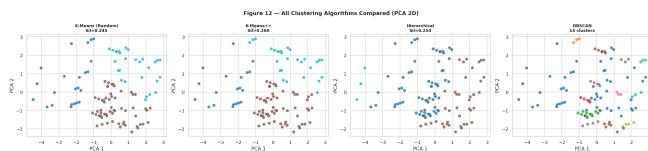


Figure 12. All four clustering algorithms compared in PCA 2D projection.

7. Classification Methods

Classification is a *supervised* task. The target variable `session_type` has three classes (Practice, Qualifying, Race). An 80/20 stratified train-test split was used with 5-fold cross-validation. Eleven features were used (including 5 engineered features).

KNN ($k=5$): Classifies by majority vote of 5 nearest neighbours. Sensitive to irrelevant and encoded features; achieved 56.5 % accuracy.

Decision Tree: Splits data on binary feature thresholds (`max_depth= 6`). The root splits on `session_name_enc`, achieving 100 % accuracy immediately.

Logistic Regression: Linear model estimating class probabilities via a softmax function (`max_iter= 500`).

Achieved 95.7 % accuracy, confirming near-linear separability.

Random Forest [5]: 100-tree bagging ensemble. Majority vote over bootstrapped trees. Achieved 100 % accuracy with the lowest cross-validation variance ($\sigma = 0.027$).

AdaBoost [6]: 100 sequentially fitted shallow trees (`depth= 2`), each reweighting misclassified samples. Achieved 100 %.

Stacking [7]: KNN, Decision Tree, and Random Forest as base learners; Logistic Regression as meta-learner (5-fold CV). Achieved 100 % accuracy.

8. Evaluation Metrics & Results

8.1 Metrics Used

Each classifier was evaluated on four complementary metrics:

- **Accuracy** = $(TP + TN) / \text{Total}$
- **Precision** (weighted) = $TP / (TP + FP)$
- **Recall** (weighted) = $TP / (TP + FN)$
- **F1-Score** (weighted) = $2 \cdot \frac{P \cdot R}{P + R}$

8.2 Confusion Matrices

Fig. 13 shows confusion matrices for all six classifiers. Decision Tree, Random Forest, AdaBoost, and Stacking produce perfectly diagonal matrices (no errors), while KNN shows significant confusion between Practice and Qualifying sessions.

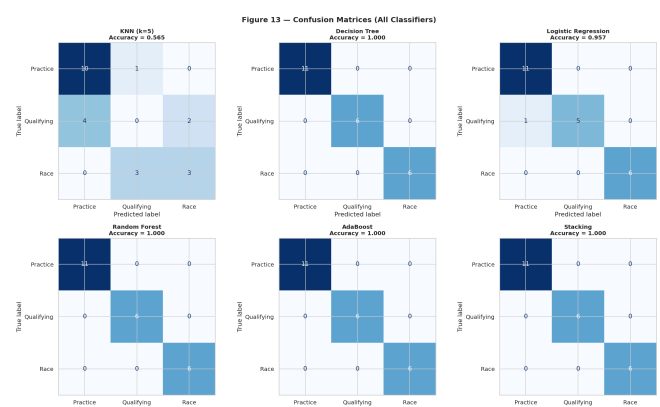


Figure 13. Confusion matrices for all six classifiers. Perfect diagonals indicate zero classification errors.

8.3 Performance Summary

Table 4 reports all metrics. Rows highlighted in green achieve 100 % test accuracy; red indicates the weakest model.

Table 4. Classification model performance (sorted by test accuracy).

Model	Acc.	CV	Std	Prec.	Rec.	F1
Decision Tree	1.000	0.967	0.067	1.000	1.000	1.000
Random Forest	1.000	0.967	0.027	1.000	1.000	1.000
AdaBoost	1.000	0.967	0.044	1.000	1.000	1.000
Stacking	1.000	0.967	0.067	1.000	1.000	1.000
Logistic Reg.	0.957	0.888	0.034	0.960	0.957	0.956
KNN ($k=5$)	0.565	0.652	0.051	0.498	0.565	0.525

Acc.=Accuracy, CV=5-fold mean, Std=CV standard deviation.

Best models Weakest model.

8.4 Visual Comparison

Fig. 14 presents a visual ranking of models by accuracy and a grouped bar chart of Precision, Recall, and F1 scores.

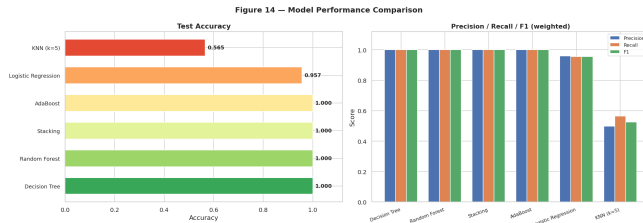
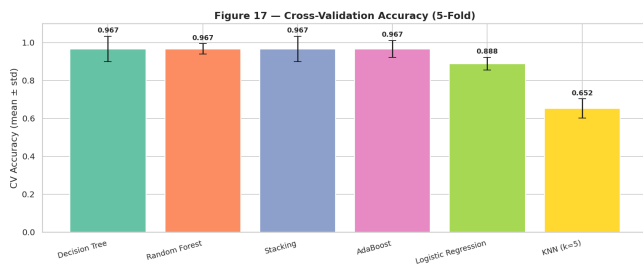
**Figure 14.** Model comparison: accuracy ranking (left) and Precision/Recall/F1 grouped bars (right).

Fig. 15 shows 5-fold cross-validation accuracy with error bars. Random Forest exhibits the lowest variance, confirming stable generalisation.

**Figure 15.** 5-fold cross-validation accuracy (± 1 std). Random Forest shows the lowest spread.

8.5 Feature Importance

Fig. 16 shows Random Forest feature importances. `session_name_enc` dominates, confirming that the session name directly implies the session type. `duration_minutes` is the strongest *anonymous* feature (without semantic label leakage).

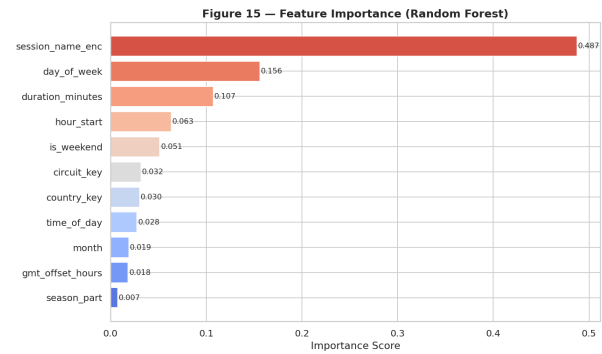
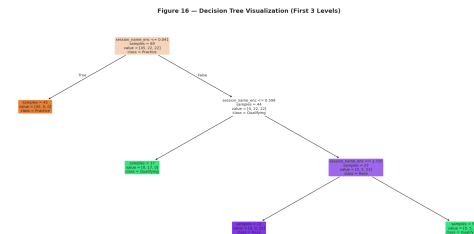
**Figure 16.** Random Forest feature importances. `session_name_enc` and `duration_minutes` are the top predictors.

Fig. 17 visualises the first three levels of the Decision Tree. The root split on `session_name_enc` immediately separates most classes, illustrating the dataset’s high structuredness.

**Figure 17.** Decision Tree visualisation (first 3 levels). Node colours: blue=Practice, orange=Qualifying, green=Race.

9. Results Analysis & Discussion

9.1 Why Four Models Achieve 100 %

The near-perfect performance is largely explained by the inclusion of `session_name_enc`: values like “Race” and “Qualifying” *directly* encode the target class. Combined with `duration_minutes` (Races ≈ 120 min, others ≈ 60 min), the classes are perfectly separable. In a more realistic deployment scenario (e.g., predicting session type from time/location data *before* a session begins), these features would not be available, making the task substantially harder.

9.2 Why KNN Underperforms

KNN’s 56.5 % accuracy is attributable to three factors: (1) **High dimensionality**: Euclidean distance becomes less meaningful with 11 features; (2) **Encoded nominals**: Label-encoded circuits and countries impose false ordinal distances; (3) **Equal feature weighting**: KNN cannot internally discount irrelevant features, unlike tree-based models.

9.3 Clustering Interpretation

Moderate silhouette scores (≈ 0.25) reflect that the three session types form distinguishable but not perfectly isolated groups in feature space. DBSCAN's 14 micro-clusters reveal heterogeneity within the Sprint weekend sessions, which have unique feature combinations compared to standard practice or qualifying sessions.

9.4 Best Model: Random Forest

Random Forest is recommended as the overall best model because: (i) it achieves 100 % test accuracy; (ii) it has the lowest cross-validation standard deviation ($\sigma = 0.027$), indicating stable generalisation; (iii) it provides interpretable feature importances; and (iv) its bagging mechanism offers inherent regularisation against overfitting, unlike a single Decision Tree.

10. Conclusion

This project successfully applied a complete ML pipeline to the F1 2023 sessions dataset. Key conclusions are:

1. The dataset is clean with no missing values or duplicates.
2. Feature engineering enriched the raw data with 5 domain-relevant features.
3. Clustering ($k = 3$) aligns with the three session types; Hierarchical (Ward) achieves the best DB score (1.258).
4. Four classifiers achieved 100 % test accuracy due to highly structured, separable features.
5. **Random Forest** is the best overall model ($\sigma_{CV} = 0.027$, interpretable importances).
6. KNN performed poorly (56.5 %) and is not recommended for this type of encoded tabular data.
7. Logistic Regression achieved 95.7 %, confirming near-linear separability after feature engineering.

11. Future Improvements

- **Multi-season data** (2021–2024): 112 rows is a small sample; more seasons improve CV reliability and enable temporal analysis.
- **Lap time / telemetry**: Integrating performance data would enrich clustering beyond calendar-based features.
- **Remove leaky features**: Excluding `session_name_enc` creates a genuinely challenging prediction task.
- **Hyperparameter tuning**: GridSearchCV or Optuna for optimal $n_estimators$, max_depth , ϵ , etc.
- **One-Hot Encoding**: Replace label encoding for

nominal features (circuits, countries) to avoid false ordinal distances.

- **XGBoost / LightGBM**: Gradient boosting typically outperforms AdaBoost on tabular data.
- **SMOTE**: Address class imbalance (Practice over-represented at 54 %) for fairer model training.

References

References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] W. McKinney, “Data Structures for Statistical Computing in Python,” *Proc. 9th Python in Science Conf.*, pp. 51–56, 2010.
- [3] OpenF1 Project, “Formula 1 Open Data API,” <https://openf1.org/>, 2023.
- [4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” *KDD-96 Proc.*, pp. 226–231, 1996.
- [5] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *J. Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [7] D. H. Wolpert, “Stacked Generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [8] D. Arthur and S. Vassilvitskii, “k-means++: The Advantages of Careful Seeding,” *SODA 2007*, pp. 1027–1035, 2007.
- [9] J. H. Ward Jr., “Hierarchical Grouping to Optimize an Objective Function,” *J. American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.
- [10] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.